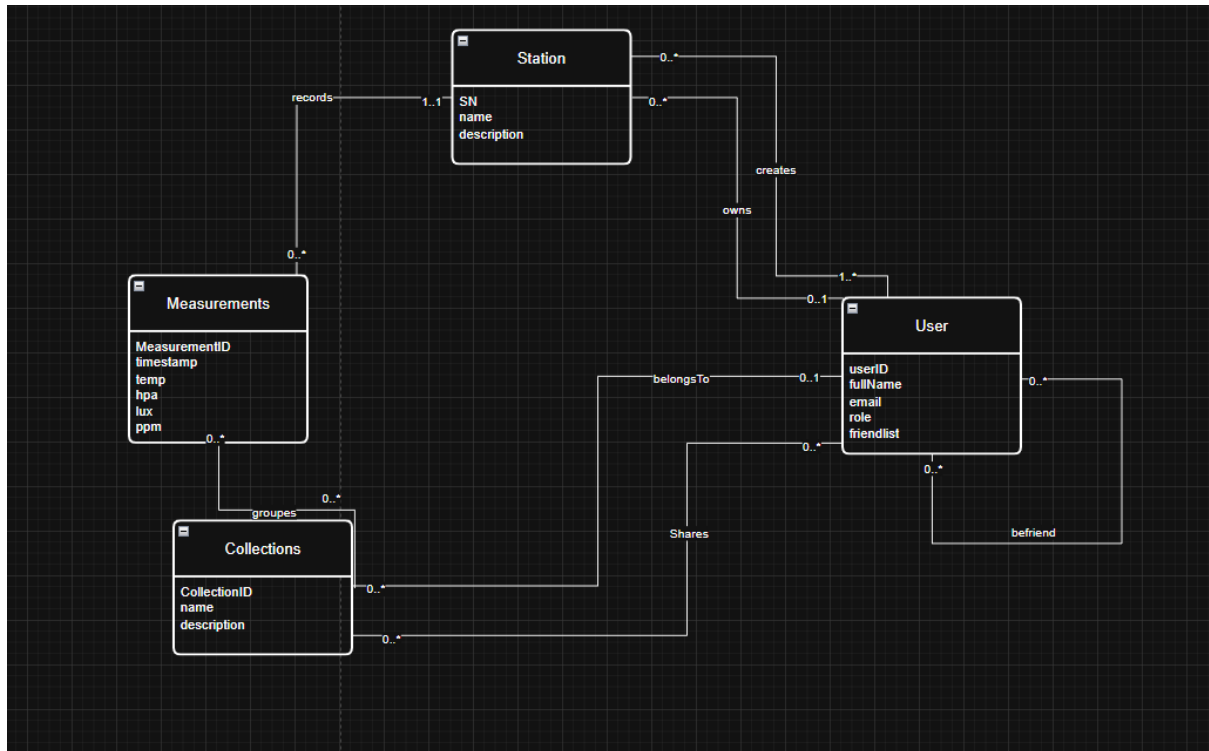


Second sprint

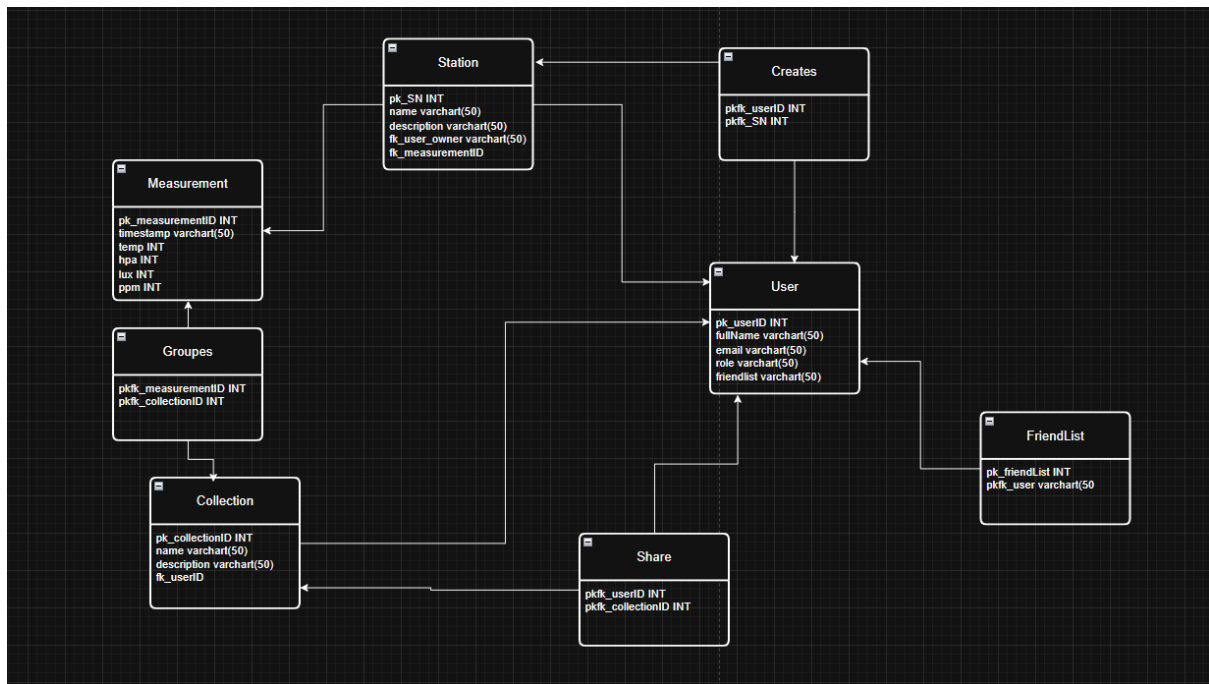
Designing the database

I started by doing the cdm based on the text in the worksheet.



For my CDM, I started by identifying the main entities from the requirements: **User**, **Station**, **Measurement**, and **Collection**.

I included all the required attributes in each entity, like the serial number for stations, timestamp and sensor data for measurements, and role information for users to distinguish between normal users and administrators. The CDM focuses on *what* data needs to be stored and *how* entities relate to each other, without worrying about implementation details."



Logical Data Model (LDM) Explanation:

For my LDM, I converted the conceptual model into a structure that can actually be implemented in a database.

The main changes I made were:

1. **Added Primary Keys** - Every table now has a unique identifier (pk_userID, pk_measurementID, etc.) so we can uniquely identify each record
2. **Added Foreign Keys** - To implement the relationships from the CDM, I added foreign keys. For example, Station has fk_user_owner to link to the User table, and Measurement has fk_stationID to link to Station
3. **Created Junction Tables** for many-to-many relationships:
 - o **Groupes** - connects Collections and Measurements (since this is M:N)
 - o **Share** - connects Users to Collections they have access to (M:N)
 - o **FriendList** - connects Users to their friends (M:N self-referencing)

These junction tables have composite primary keys made up of the foreign keys from both related tables.

4. **Specified Data Types** - I chose appropriate data types and sizes: VARCHAR for text fields with specified lengths, INT for numeric IDs and sensor readings, and I made sure to match the requirements (temperature in °C, humidity in %, etc.)

5. **Defined Constraints** - I added rules like NOT NULL for required fields, UNIQUE for usernames and emails, and CHECK constraints for the role field to ensure data integrity.

Implimenting the database

Then i moved on to creating the database it is nothing special , the LDM help a lot with doing that since it acts as a blueprint.

```
DROP TABLE IF EXISTS FriendList,Share,Groupes,Measurement,Station,Collection,User;

CREATE TABLE User (
  pk_userID INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL UNIQUE,
  fullName VARCHAR(90) NOT NULL,
  email VARCHAR(50) NOT NULL UNIQUE,
  role VARCHAR(50) NOT NULL DEFAULT 'user',
  friendlist VARCHAR(50),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT chk_role CHECK (role IN ('user','admin'))
);

CREATE TABLE Station (
  pk_SN_INT INT AUTO_INCREMENT PRIMARY KEY,
  SN VARCHAR(50) NOT NULL UNIQUE,
  name VARCHAR(50),
  description VARCHAR(50),
  fk_user_owner VARCHAR(50) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (fk_user_owner) REFERENCES User(username) ON DELETE RESTRICT ON
  UPDATE CASCADE
);

CREATE TABLE Measurement (
  pk_measurementID INT AUTO_INCREMENT PRIMARY KEY,
  timestamp VARCHAR(50) NOT NULL,
  temp INT NOT NULL,
  hpa INT NOT NULL,
  lux INT NOT NULL,
  ppm INT NOT NULL,
  humidity DECIMAL(5,2),
  fk_stationID INT NOT NULL,
  recorded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (fk_stationID) REFERENCES Station(pk_SN_INT) ON DELETE CASCADE ON
  UPDATE CASCADE
);
```

```

CREATE TABLE Collection (
    pk_collectionID INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    description VARCHAR(50),
    fk_userID INT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (fk_userID) REFERENCES User(pk_userID) ON DELETE CASCADE ON
UPDATE CASCADE
);

CREATE TABLE Groupes (
    pkfk_measurementID INT NOT NULL,
    pkfk_collectionID INT NOT NULL,
    added_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (pkfk_measurementID,pkfk_collectionID),
    FOREIGN KEY (pkfk_measurementID) REFERENCES Measurement(pk_measurementID) ON
DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (pkfk_collectionID) REFERENCES Collection(pk_collectionID) ON DELETE
CASCADE ON UPDATE CASCADE
);

CREATE TABLE Share (
    pkfk_userID INT NOT NULL,
    pkfk_collectionID INT NOT NULL,
    shared_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (pkfk_userID,pkfk_collectionID),
    FOREIGN KEY (pkfk_userID) REFERENCES User(pk_userID) ON DELETE CASCADE ON
UPDATE CASCADE,
    FOREIGN KEY (pkfk_collectionID) REFERENCES Collection(pk_collectionID) ON DELETE
CASCADE ON UPDATE CASCADE
);

CREATE TABLE FriendList (
    pk_friendList_INT INT AUTO_INCREMENT PRIMARY KEY,
    pkfk_user1 VARCHAR(50) NOT NULL,
    pkfk_user2 VARCHAR(50) NOT NULL,
    friendship_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (pkfk_user1) REFERENCES User(username) ON DELETE CASCADE ON
UPDATE CASCADE,
    FOREIGN KEY (pkfk_user2) REFERENCES User(username) ON DELETE CASCADE ON
UPDATE CASCADE,
    CONSTRAINT chk_no_self_friend CHECK (pkfk_user1 != pkfk_user2),
    UNIQUE (pkfk_user1,pkfk_user2)
);

CREATE INDEX idx_station_owner ON Station(fk_user_owner);
CREATE INDEX idx_measurement_station ON Measurement(fk_stationID);
CREATE INDEX idx_measurement_timestamp ON Measurement(timestamp);
CREATE INDEX idx_collection_creator ON Collection(fk_userID);
CREATE INDEX idx_user_email ON User(email);

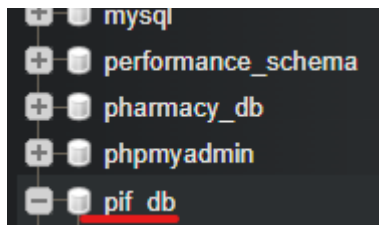
```

Inserting Dummy data into the DB

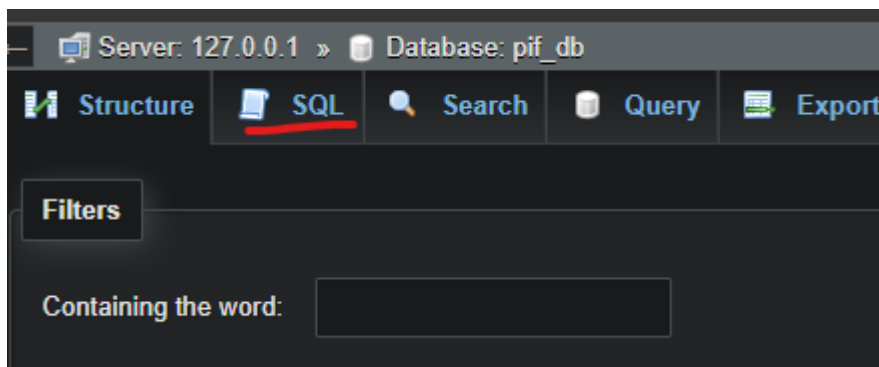
For this task I am going to use an AI tool called calude.ai.

I gave the ai tool context about the whole project and it gave the dummy data as an sql code.

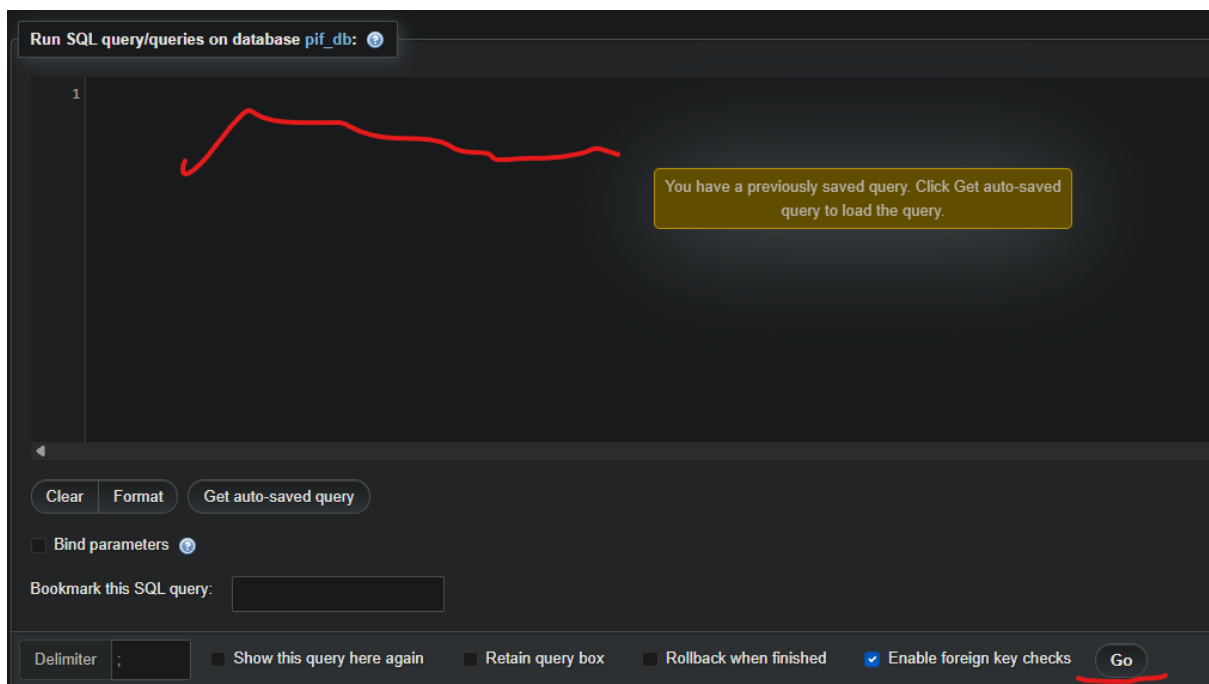
I will use phpMyAdmin to work on the db throughout the project.



As you can see in the picture above i have created a pif_db.



Go to sql.



Paste your sql code here and click on go.

This is the sql code for the dummy data.

```
-- Sample Data for Station Management System

-- Users (2 admins, 3 regular users)
INSERT INTO User (username,fullName,email,role) VALUES
('admin_john','John Anderson','john@station.com','admin'),
('admin_sarah','Sarah Mitchell','sarah@station.com','admin'),
('mike_chen','Michael Chen','mike@email.com','user'),
('emma_wilson','Emma Wilson','emma@email.com','user'),
('alex_garcia','Alex Garcia','alex@email.com','user');

-- Stations (5 stations)
INSERT INTO Station (SN,name,description,fk_user_owner) VALUES
('SN-1001','Home Weather','Living room monitoring','mike_chen'),
('SN-1002','Greenhouse','Temperature control','mike_chen'),
('SN-1003','Office Tracker','Office monitoring','emma_wilson'),
('SN-2104','Garden Station','Outdoor conditions','alex_garcia'),
('SN-2105','Basement Sensor','Moisture tracking','alex_garcia');

-- Measurements (15 measurements)
INSERT INTO Measurement (timestamp,temp,hpa,lux,ppm,humidity,fk_stationID) VALUES
('2024-11-18 08:00:00',22,1013,450,420,45.5,1),
('2024-11-18 12:00:00',24,1012,850,435,43.2,1),
('2024-11-18 18:00:00',23,1014,320,410,46.8,1),
('2024-11-18 07:00:00',28,1010,1200,580,68.5,2),
('2024-11-18 11:00:00',32,1009,2500,620,72.3,2),
('2024-11-18 15:00:00',34,1008,3200,650,75.1,2),
('2024-11-18 09:00:00',23,1014,550,480,40.5,3),
('2024-11-18 13:00:00',24,1013,720,520,39.2,3),
('2024-11-18 17:00:00',24,1013,650,510,38.8,3),
('2024-11-18 06:00:00',15,1015,5200,380,78.5,4),
('2024-11-18 12:00:00',19,1014,8500,390,65.2,4),
('2024-11-18 18:00:00',17,1016,2100,385,72.0,4),
('2024-11-18 08:00:00',18,1020,85,520,62.5,5),
('2024-11-18 14:00:00',18,1021,90,525,63.2,5),
('2024-11-18 20:00:00',17,1021,45,510,64.5,5);

-- Collections (4 collections)
INSERT INTO Collection (name,description,fk_userID) VALUES
('Home Data','Home weather patterns',3),
('Greenhouse Cycle','Growing conditions',3),
('Office Comfort','Office environment',4),
('Garden Seasonal','Seasonal monitoring',5);

-- Groupes (measurement-collection links)
INSERT INTO Groupes (pkfk_measurementID,pkfk_collectionID) VALUES
(1,1),(2,1),(3,1),(4,2),(5,2),(6,2),(7,3),(8,3),(9,3),(10,4),(11,4),(12,4);

-- Friendships
INSERT INTO FriendList (pkfk_user1,pkfk_user2) VALUES
('mike_chen','emma_wilson'),
('mike_chen','alex_garcia'),
('emma_wilson','alex_garcia');

-- Shared Collections
INSERT INTO Share (pkfk_userID,pkfk_collectionID) VALUES
(4,1),(4,2),(3,3),(5,4),(3,4);

-- Verification
SELECT 'Users:' AS Info,COUNT(*) AS Count FROM User
UNION ALL SELECT 'Stations:',COUNT(*) FROM Station
UNION ALL SELECT 'Measurements:',COUNT(*) FROM Measurement
UNION ALL SELECT 'Collections:',COUNT(*) FROM Collection
UNION ALL SELECT 'Friendships:',COUNT(*) FROM FriendList
UNION ALL SELECT 'Shares:',COUNT(*) FROM Share;
```